

Pedidos

Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén Sánchez, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

1. Requisitos de información (RI)	2
1.1. RI-1: Usuarios	2
1.2. RI-2: Productos	2
1.3. RI-3: Pedidos	2
2. Reglas de negocio (RN)	4
2.1. RN-1: Límite de pedidos	4
2.2. RN-2: Agosto inhábil	4
2.3. RN-3: Stock suficiente	4
3. Pruebas de aceptación (PA)	5
3.1. PA-1: Pedidos	5
4. Diagrama de clases	6
5. Álgebra relacional	8
6. Script SQL para crear la base de datos	13
7. Script SQL para la carga inicial de datos	15
8. Consultas	17
9. Procedimientos	19
10. Pruebas SQL	23

Requisitos

Requisitos de información (RI)

1.1. RI-1: Usuarios

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los usuarios: nombre, provincia y fecha de alta en el Sistema de Pedidos.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico

1.2. RI-2: Productos

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los productos: descripción, precio y stock en el almacén.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico

1.3. RI-3: Pedidos

- Como: Profesor de la asignatura

1. Requisitos de información (RI)

- Quiero: Poder almacenar la siguiente información de los pedidos: fecha de compra en la que el usuario hace el pedido y cantidad de productos que se piden.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico

Reglas de negocio (RN)

2.1. RN-1: Límite de pedidos

- Como: Profesor de la asignatura
- Quiero: Que un usuario no pueda hacer más de tres pedidos al día
- Para: Que el estudiante practique con restricciones simples

2.2. RN-2: Agosto inhábil

- Como: Profesor de la asignatura
- Quiero: Que no se puedan hacer pedidos en el mes de agosto
- Para: Que el estudiante practique con restricciones simples

2.3. RN-3: Stock suficiente

- Como: Profesor de la asignatura
- Quiero: Que solo se pueda realizar el pedido si hay stock suficiente
- Para: Que el estudiante practique con restricciones simples

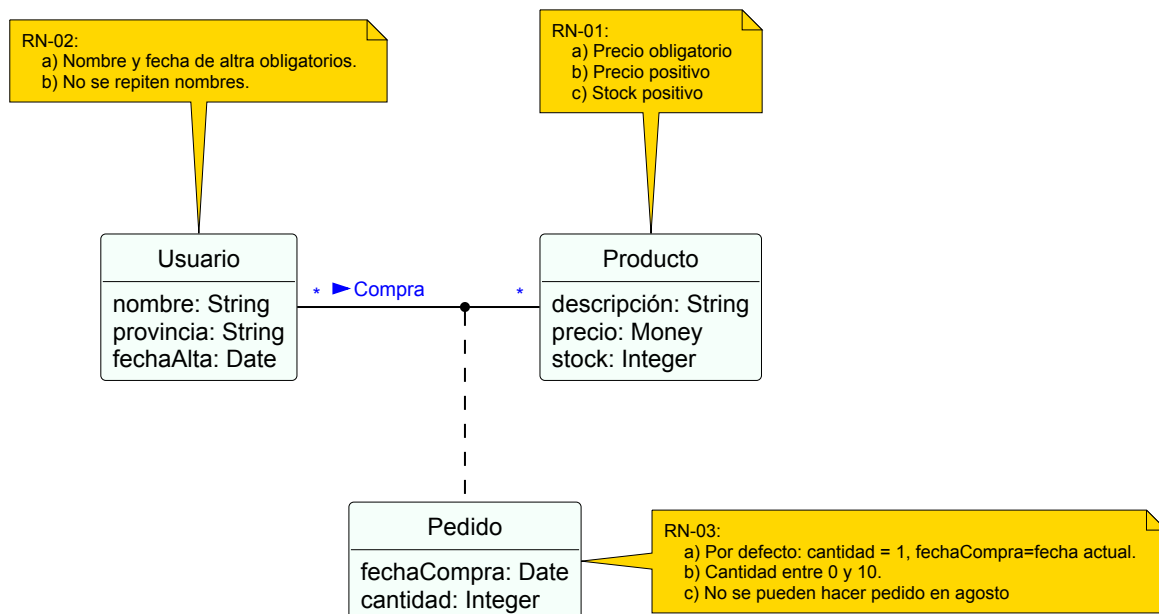
Pruebas de aceptación (PA)

3.1. PA-1: Pedidos

1. ☒ Crear un nuevo pedido con todos los datos correctos según las reglas de negocio.
2. ☐ Crear un nuevo pedido sin especificar la cantidad de productos.
3. ☐ Crear un nuevo pedido sin especificar la fecha de compra.
4. ☐ Crear un nuevo pedido con más de tres pedidos al día.
5. ☐ Crear un nuevo pedido en el mes de agosto.
6. ☐ Crear un nuevo pedido sin stock suficiente.

Modelo Conceptual

Diagrama de clases



Modelo Relacional

```

Usuarios = { usuarioId, nombre, provincia, fechaAlta }
  PK(usuarioId)
Productos = { productoId, descripcion, precio, stock }
  PK(productoId)
Pedidos = { pedidoId, usuarioId, productoId, fechaCompra, cantidad }
  PK(pedidoId)
  FK(usuarioId) / Usuarios
  FK(productoId) / Productos
  
```

4. Diagrama de clases

```
(u1, "David Ruiz", "Sevilla", "2018-05-18"),
(u2, "Marta López", "Málaga", "2018-06-12"),
(u3, "Raquel Lobato", "Granada", "2018-12-01"),
(u4, "Antonio Gómez", "Sevilla", "2018-03-11"),
(u5, "Inma Hernández", "Málaga", "2018-04-12"),
(u6, "Jimena Martín", "Granada", "2018-05-13"),
(u7, "Carlos Rivero", "Huelva", "2018-09-07"),
(u8, "Carlos Arévalo", "Málaga", "2018-09-07")
}
Productos = {
  (p1, "Mi Band 3", 19.90, 50),
  (p2, "Mi Band 4", 29.90, 20),
  (p3, "Pulsera compatible con Mi Band 3 y 4", 9.90, 150),
  (p4, "Mi Scooter", 349.90, 25),
  (p5, "Rueda trasera de respuesto Mi Scooter", 19.90, 50),
  (p6, "Rueda delantera de respuesto Mi Scooter", 59.90, 50)
}
Pedidos = {
  (pe1, u1, p1, "2019-05-13", 2),
  (pe2, u1, p3, "2019-05-13", 2),
  (pe3, u2, p2, "2019-06-11", 3),
  (pe4, u2, p3, "2019-06-11", 1),
  (pe5, u3, p4, "2019-06-15", 2),
  (pe6, u4, p5, "2019-06-18", 1),
  (pe7, u4, p6, "2019-06-18", 1),
  (pe8, u5, p4, "2019-12-15", 2),
  (pe9, u7, p1, "2019-12-15", 1),
  (pe10, u7, p2, "2019-12-16", 1),
  (pe11, u7, p3, "2019-12-17", 1),
  (pe12, u7, p4, "2019-12-18", 1),
  (pe13, u7, p5, "2019-12-19", 1),
  (pe14, u7, p6, "2019-12-20", 1),
  (pe15, u8, p1, "2019-12-15", 1)
}
```

Álgebra relacional

Renombrado:

$\rho_{U(uId,n,prov,fa)} Usuarios(usuarioId, nombre, provincia, fechaAlta)$

$\rho_{P(pId,d,pr,st)} Productos(productoId, descripcion, precio, stock)$

$\rho_{Ped(pedId,uId,pId,fc,c)} Pedidos(pedidoId, usuarioId, productoId, fechaCompra, cantidad)$

Listados y consultas:

- **Todos los pedidos con sus usuarios y productos:** Join natural entre las tres tablas para obtener información completa de cada pedido.

$$UPP \leftarrow U \bowtie Ped \bowtie P$$

```
UPP = { uId, n, prov, fa, pedId, pId, fc, c, d, pr, st }
```

```
UPP = {
  (u1, "David Ruiz", "Sevilla", "2018-05-18", pe1, p1, "2019-05-13", 2, "Mi Band 3", 19.90,
  50),
  (u1, "David Ruiz", "Sevilla", "2018-05-18", pe2, p3, "2019-05-13", 2, "Pulsera compatible
  con Mi Band 3 y 4", 9.90, 150),
  (u2, "Marta López", "Málaga", "2018-06-12", pe3, p2, "2019-06-11", 3, "Mi Band 4", 29.90,
  20),
  (u2, "Marta López", "Málaga", "2018-06-12", pe4, p3, "2019-06-11", 1, "Pulsera compatible
  con Mi Band 3 y 4", 9.90, 150),
  (u3, "Raquel Lobato", "Granada", "2018-12-01", pe5, p4, "2019-06-15", 2, "Mi Scooter",
  349.90, 25),
  (u4, "Antonio Gómez", "Sevilla", "2018-03-11", pe6, p5, "2019-06-18", 1, "Rueda trasera de
```

```

    respuesta Mi Scooter", 19.90, 50),
    (u4, "Antonio Gómez", "Sevilla", "2018-03-11", pe7, p6, "2019-06-18", 1, "Rueda delantera de
    respuesta Mi Scooter", 59.90, 50),
    (u5, "Inma Hernández", "Málaga", "2018-04-12", pe8, p4, "2019-12-15", 2, "Mi Scooter",
    349.90, 25),
    (u7, "Carlos Rivero", "Huelva", "2018-09-07", pe9, p1, "2019-12-15", 1, "Mi Band 3", 19.90,
    50),
    (u7, "Carlos Rivero", "Huelva", "2018-09-07", pe10, p2, "2019-12-16", 1, "Mi Band 4", 29.90,
    20),
    (u7, "Carlos Rivero", "Huelva", "2018-09-07", pe11, p3, "2019-12-17", 1, "Pulsera compatible
    con Mi Band 3 y 4", 9.90, 150),
    (u7, "Carlos Rivero", "Huelva", "2018-09-07", pe12, p4, "2019-12-18", 1, "Mi Scooter",
    349.90, 25),
    (u7, "Carlos Rivero", "Huelva", "2018-09-07", pe13, p5, "2019-12-19", 1, "Rueda trasera de
    respuesta Mi Scooter", 19.90, 50),
    (u7, "Carlos Rivero", "Huelva", "2018-09-07", pe14, p6, "2019-12-20", 1, "Rueda delantera de
    respuesta Mi Scooter", 59.90, 50),
    (u8, "Carlos Arévalo", "Málaga", "2018-09-07", pe15, p1, "2019-12-15", 1, "Mi Band 3",
    19.90, 50)
}

```

- **Pedidos de usuarios de Málaga:** Filtramos los pedidos completos (UPP) seleccionando solo aquellos cuyo usuario tiene provincia Málaga.

$$\sigma_{prov = \text{Málaga}}(UPP)$$

```

PedidosMálaga = { uId, n, prov, fa, pedId, pId, fc, c, d, pr, st }

PedidosMálaga = {
    (u2, "Marta López", "Málaga", "2018-06-12", pe3, p2, "2019-06-11", 3, "Mi Band 4", 29.90,
    20),
    (u2, "Marta López", "Málaga", "2018-06-12", pe4, p3, "2019-06-11", 1, "Pulsera compatible
    con Mi Band 3 y 4", 9.90, 150),
    (u5, "Inma Hernández", "Málaga", "2018-04-12", pe8, p4, "2019-12-15", 2, "Mi Scooter",
    349.90, 25),
    (u8, "Carlos Arévalo", "Málaga", "2018-09-07", pe15, p1, "2019-12-15", 1, "Mi Band 3",
    19.90, 50)
}

```

- **Descripción y stock de productos con stock < 100:** Proyectamos solo la descripción y stock de aquellos productos cuyo stock es inferior a 100 unidades.

$$\Pi_{d, st} \left(\sigma_{st < 100}(P) \right)$$

```

StockBajo = { d, st }

StockBajo = {
    ("Mi Band 3", 50),
    ("Mi Band 4", 20),
    ("Mi Scooter", 25),
    ("Rueda trasera de respuesta Mi Scooter", 50),
    ("Rueda delantera de respuesta Mi Scooter", 50)
}

```

- **Número de pedidos por usuario:** Agrupamos por usuario (*uId*) y contamos cuántos pedidos tiene cada uno. El join con *U* asegura incluir a todos los usuarios que han hecho pedido.

$$uId, n, \rho_{total}(\text{COUNT}(pedId)) \gamma uId, n(U \bowtie Ped)$$

```
NumPedidosUsuario = { uId, n, total }
```

```
NumPedidosUsuario = {
  (u1, "David Ruiz", 2),
  (u2, "Marta López", 2),
  (u3, "Raquel Lobato", 1),
  (u4, "Antonio Gómez", 2),
  (u5, "Inma Hernández", 1),
  (u7, "Carlos Rivero", 6),
  (u8, "Carlos Arévalo", 1)
}
```

- **Importe total por usuario:** Agrupamos por usuario y sumamos el importe de todos sus pedidos (precio × cantidad). Solo incluye usuarios con pedidos.

$$uId, n, \rho_{totalGasto}(\text{SUM}(pr \cdot c)) \gamma uId, n(UPP)$$

```
ImporteTotalUsuario = { uId, n, totalGasto }
```

```
ImporteTotalUsuario = {
  (u1, "David Ruiz", 59.60),
  (u2, "Marta López", 99.60),
  (u3, "Raquel Lobato", 699.80),
  (u4, "Antonio Gómez", 79.80),
  (u5, "Inma Hernández", 699.80),
  (u7, "Carlos Rivero", 489.40),
  (u8, "Carlos Arévalo", 19.90)
}
```

- **Pedidos por mes:** Agrupamos por mes (extrayendo el mes de la fecha de compra) y contamos cuántos pedidos hubo en cada mes.

$$MES(fc), \rho_{total}(\text{COUNT}(pedId)) \gamma MES(fc)(Ped)$$

```
PedidosPorMes = { mes, total }
```

```
PedidosPorMes = {
  (5, 2),
  (6, 5),
  (12, 8)
}
```

- **Usuario que más gasta cada mes:** Primero calculamos el gasto total de cada usuario por mes. Luego obtenemos el máximo gasto de cada mes. Finalmente, filtramos para quedarnos solo con los usuarios cuyo gasto coincide con el máximo de su mes.

$$GPUM \leftarrow uId, n, \text{MES}(fc), \rho_{totalGasto}(\text{SUM}(pr \cdot c)) \gamma uId, n, \text{MES}(fc)(UPP)$$

```
GPUM = { uId, n, mes, totalGasto }

GPUM = {
  (u1, "David Ruiz", 5, 59.60),
  (u2, "Marta López", 6, 99.60),
  (u3, "Raquel Lobato", 6, 699.80),
  (u4, "Antonio Gómez", 6, 79.80),
  (u5, "Inma Hernández", 12, 699.80),
  (u7, "Carlos Rivero", 12, 489.40),
  (u8, "Carlos Arévalo", 12, 19.90)
}
```

$$MGPM \leftarrow \gamma_{mes, \rho_{maxGasto}(\text{MAX}(totalGasto))} mes(GPUM)$$

```
MGPM = { mes, maxGasto }

MGPM = {
  (5, 59.60),
  (6, 699.80),
  (12, 699.80)
}
```

$$\Pi_{n, mes, totalGasto} \left(\sigma_{totalGasto = maxGasto} (GPUM \bowtie MGPM) \right)$$

```
MasGastaPorMes = { n, mes, totalGasto }

MasGastaPorMes = {
  ("David Ruiz", 5, 59.60),
  ("Raquel Lobato", 6, 699.80),
  ("Inma Hernández", 12, 699.80)
}
```

- **Mes de máxima recaudación:** Primero calculamos la recaudación total de cada mes (sumando el importe de todos los pedidos). Luego obtenemos el mes con mayor recaudación.

$$RPM \leftarrow \gamma_{MES(fc), \rho_{recaudacion}(\text{SUM}(pr \cdot c))} MES(fc)(UPP)$$

```
RPM = { mes, recaudacion }

RPM = {
  (5, 59.60),
  (6, 879.20),
  (12, 1209.10)
}
```

$$MaxRecaudacion \leftarrow \gamma_{\rho_{maxRec}(\text{MAX}(recaudacion))} (RPM)$$

```
MaxRecaudacion = { maxRec }

MaxRecaudacion = {
  (1209.10)
}
```

$$\prod_{mes, recaudacion} \left(\sigma_{recaudacion=maxRec} (RPM \times MaxRecaudacion) \right)$$

```
MesMaxRecaudacion = { mes, recaudacion }
```

```
MesMaxRecaudacion = {  
  (12, 1209.10)  
}
```

Modelo Tecnológico

Script SQL para crear la base de datos

pedidos/createDB.sql

```
--
-- Autores: Inma Hernández, David Ruiz, Daniel Ayala
-- Fecha: Noviembre 2025
-- Descripción: Script para crear la BD del ejercicio de Pedidos
--

DROP DATABASE IF EXISTS OrdersDB;
CREATE DATABASE OrdersDB;
USE OrdersDB;

-- =====
-- Eliminamos tablas previas (orden inverso de dependencias)
-- =====
SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS users;
SET FOREIGN_KEY_CHECKS = 1;

-- =====
-- Tabla: users
-- =====
CREATE TABLE users (
```

```

user_id INT AUTO_INCREMENT,
full_name VARCHAR(63) NOT NULL,
province VARCHAR(63),
start_date DATE NOT NULL,
PRIMARY KEY (user_id),
CONSTRAINT rn01_users_name_not_null CHECK (full_name IS NOT NULL),
CONSTRAINT rn02_users_name_unique UNIQUE (full_name)
);

-- =====
-- Tabla: products
-- =====
CREATE TABLE products (
    product_id INT AUTO_INCREMENT,
    description VARCHAR(128),
    price DECIMAL(8,2) NOT NULL,
    stock INT,
    PRIMARY KEY (product_id),
    CONSTRAINT rn03_products_price_not_null CHECK (price IS NOT NULL),
    CONSTRAINT rn04_products_price_nonnegative CHECK (price >= 0),
    CONSTRAINT rn05_products_stock_nonnegative CHECK (stock >= 0)
);

-- =====
-- Tabla: orders
-- =====
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    amount INT DEFAULT 1,
    purchase_date DATE DEFAULT (CURRENT_DATE),
    PRIMARY KEY (order_id),
    CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES
users(user_id),
    CONSTRAINT fk_orders_product FOREIGN KEY (product_id) REFERENCES
products(product_id),
    CONSTRAINT rn06_orders_amount_not_null CHECK (amount IS NOT NULL),
    CONSTRAINT rn07_orders_amount_range CHECK (amount BETWEEN 1 AND 10),
    CONSTRAINT rn08_orders_month_not_august CHECK (MONTH(purchase_date) <>
8)
);

```

Script SQL para la carga inicial de datos

pedidos/populateDB.sql

```
--
-- Autores: Inma Hernández, David Ruiz, Daniel Ayala
-- Fecha: Noviembre 2025
-- Descripción: Procedimiento para poblar la BD de Pedidos
--
USE OrdersDB;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_populate()
BEGIN
    DELETE FROM orders;
    DELETE FROM products;
    DELETE FROM users;

    ALTER TABLE users AUTO_INCREMENT = 1;
    ALTER TABLE products AUTO_INCREMENT = 1;
    ALTER TABLE orders AUTO_INCREMENT = 1;

    INSERT INTO users (full_name, province, start_date) VALUES
        ('David Ruiz', 'Sevilla', '2018-05-18'),
        ('Marta López', 'Málaga', '2018-06-12'),
        ('Raquel Lobato', 'Granada', '2018-12-01'),
        ('Antonio Gómez', 'Sevilla', '2018-03-11'),
```

```

('Inma Hernández', 'Málaga', '2018-04-12'),
('Jimena Martín', 'Granada', '2018-05-13'),
('Carlos Rivero', 'Huelva', '2018-09-07'),
('Carlos Arévalo', 'Málaga', '2018-09-07');

INSERT INTO products (description, price, stock) VALUES
('Mi Band 3', 19.90, 50),
('Mi Band 4', 29.90, 20),
('Pulsera compatible con Mi Band 3 y 4', 9.90, 150),
('Mi Scooter', 349.90, 25),
('Rueda trasera de respuesta Mi Scooter', 19.90, 50),
('Rueda delantera de respuesta Mi Scooter', 59.90, 50);

INSERT INTO orders (user_id, product_id, amount, purchase_date) VALUES
(1, 1, 2, '2019-05-13'),
(1, 3, 2, '2019-05-13'),
(2, 2, 3, '2019-06-11'),
(2, 3, 1, '2019-06-11'),
(3, 4, 2, '2019-06-15'),
(4, 5, 1, '2019-06-18'),
(4, 6, 1, '2019-06-18'),
(5, 4, 2, '2019-12-15'),
(7, 1, 1, '2019-12-15'),
(7, 2, 1, '2019-12-16'),
(7, 3, 1, '2019-12-17'),
(7, 4, 1, '2019-12-18'),
(7, 5, 1, '2019-12-19'),
(7, 6, 1, '2019-12-20'),
(8, 1, 1, '2019-12-15');

END //
DELIMITER ;

CALL p_populate();

```

Consultas

pedidos/queries.sql

```
--
-- Autores: Inma Hernández, David Ruiz, Daniel Ayala
-- Fecha: Noviembre 2025
-- Descripción: Consultas de referencia para Pedidos
--

USE OrdersDB;

-- Usuarios de Sevilla
SELECT * FROM users WHERE province = 'Sevilla';

-- Usuarios que no son de Sevilla
SELECT * FROM users WHERE province <> 'Sevilla';

-- Productos agotados o sin stock
SELECT * FROM products WHERE stock = 0;

-- Pedidos de un usuario concreto
SELECT * FROM orders WHERE user_id = 1;

-- Usuarios sin pedidos
SELECT *
FROM users u
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.user_id = u.user_id
);
```

```
-- Usuarios que han comprado todos los productos
SELECT u.user_id, u.full_name
FROM users u
WHERE NOT EXISTS (
    SELECT 1 FROM products p
    WHERE NOT EXISTS (
        SELECT 1 FROM orders o
        WHERE o.user_id = u.user_id AND o.product_id = p.product_id
    )
);
```

Procedimientos

pedidos/procedures.sql

```
--
-- Autores: Daniel Ayala, Inma Hernández, David Ruiz
-- Fecha: Noviembre 2025
-- Descripción: Funciones y procedimientos auxiliares para Pedidos
--

USE OrdersDB;

-- Unidades vendidas de un producto
DELIMITER //
CREATE OR REPLACE FUNCTION f_get_sold_units(p_product_id INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN (
        SELECT COALESCE(SUM(amount), 0)
        FROM orders
        WHERE product_id = p_product_id
    );
END //
DELIMITER ;

-- Importe total entre dos fechas
DELIMITER //
CREATE OR REPLACE FUNCTION f_purchase_between_dates(p_date1 DATE, p_date2
DATE)
```

```

RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN (
        SELECT COALESCE(SUM(amount * price), 0)
        FROM orders o
        JOIN products p ON p.product_id = o.product_id
        WHERE purchase_date BETWEEN p_date1 AND p_date2
    );
END //
DELIMITER ;

-- Producto más vendido en un año dado
DELIMITER //
CREATE OR REPLACE FUNCTION f_best_seller(p_year INT)
RETURNS INT
DETERMINISTIC
BEGIN
    RETURN (
        SELECT product_id
        FROM orders o
        JOIN products p ON p.product_id = o.product_id
        WHERE YEAR(purchase_date) = p_year
        GROUP BY product_id
        ORDER BY SUM(amount) DESC
        LIMIT 1
    );
END //
DELIMITER ;

-- Dinero gastado por un usuario
DELIMITER //
CREATE OR REPLACE FUNCTION f_spent_money_user(p_user_id INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    RETURN (
        SELECT COALESCE(SUM(amount * price), 0)
        FROM orders o
        JOIN products p ON p.product_id = o.product_id
        WHERE o.user_id = p_user_id
    );
END //
DELIMITER ;

-- Stock de un producto
DELIMITER //
CREATE OR REPLACE FUNCTION f_get_stock(p_product_id INT)
RETURNS INT
DETERMINISTIC

```

```

BEGIN
    RETURN (
        SELECT stock
        FROM products
        WHERE product_id = p_product_id
    );
END //
DELIMITER ;

-- Crear pedido con fecha y cantidad (defaults manejados por la tabla)
DELIMITER //
CREATE OR REPLACE PROCEDURE p_insert_order(
    IN p_user_id INT,
    IN p_product_id INT,
    IN p_amount INT,
    IN p_purchase_date DATE
)
BEGIN
    INSERT INTO orders(user_id, product_id, amount, purchase_date)
    VALUES (p_user_id, p_product_id, IFNULL(p_amount, 1),
    IFNULL(p_purchase_date, CURRENT_DATE()));
END //
DELIMITER ;

-- Cambiar precios en un porcentaje
DELIMITER //
CREATE OR REPLACE PROCEDURE p_change_prices(IN p_fraction DECIMAL(6,4))
BEGIN
    UPDATE products
    SET price = price + price * p_fraction;
END //
DELIMITER ;

-- Crear vista de pedidos de un usuario en un año
DELIMITER //
CREATE OR REPLACE PROCEDURE p_create_orders_view(IN p_full_name
VARCHAR(64), IN p_orders_year INT)
BEGIN
    DECLARE v_user_id INT;
    SELECT user_id INTO v_user_id FROM users WHERE full_name = p_full_name;
    IF v_user_id IS NOT NULL THEN
        SET @sql = CONCAT('CREATE OR REPLACE VIEW vOrders',
REPLACE(p_full_name, ' ', ''), p_orders_year, ' AS\n',
        'SELECT * FROM orders WHERE user_id = ',
v_user_id, ' AND YEAR(purchase_date) = ', p_orders_year, ';');
        PREPARE stmt FROM @sql;
        EXECUTE stmt;
        DEALLOCATE PREPARE stmt;
    END IF;
END //

```

```

DELIMITER ;

-- Crear vistas para usuarios con al menos N pedidos en un año
DELIMITER //
CREATE OR REPLACE PROCEDURE p_create_orders_views(IN p_min_orders INT, IN
p_orders_year INT)
BEGIN
    DECLARE v_full_name VARCHAR(64);
    DECLARE v_done BOOLEAN DEFAULT FALSE;

    DECLARE cur CURSOR FOR
        SELECT full_name
        FROM users u
        JOIN orders o ON o.user_id = u.user_id
        WHERE YEAR(o.purchase_date) = p_orders_year
        GROUP BY full_name
        HAVING COUNT(*) >= p_min_orders;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_full_name;
        IF v_done THEN
            LEAVE read_loop;
        END IF;
        CALL p_create_orders_view(v_full_name, p_orders_year);
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

```

Pruebas SQL

pedidos/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre 2025
-- Descripción: Tests negativos para la BD de Pedidos
--
USE OrdersDB;

-- =====
-- TABLA DE RESULTADOS
-- =====
CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- =====
-- PROCEDIMIENTO AUXILIAR
-- =====
DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(
    IN p_test_id VARCHAR(20),
    IN p_message VARCHAR(500),
    IN p_status ENUM('PASS', 'FAIL', 'ERROR')
```

```

)
BEGIN
    INSERT INTO test_results(test_id, test_name, test_message, test_status)
    VALUES (p_test_id, SUBSTRING_INDEX(p_message, ':', 1), p_message,
p_status);
END //
DELIMITER ;

-- =====
-- TESTS (RN01 - RN08)
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn01_user_name_not_null()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN01', 'RN01: full_name no puede ser NULL',
'PASS');

    CALL p_populate();

    INSERT INTO users (full_name, province, start_date) VALUES (NULL,
'Test', CURDATE());

    CALL p_log_test('RN01', 'ERROR: Se permitió full_name NULL', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn02_user_name_unique()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN02', 'RN02: full_name debe ser único', 'PASS');

    CALL p_populate();

    INSERT INTO users (full_name, province, start_date) VALUES ('Nombre
Duplicado', 'Sevilla', CURDATE());
    INSERT INTO users (full_name, province, start_date) VALUES ('Nombre
Duplicado', 'Huelva', CURDATE());

    CALL p_log_test('RN02', 'ERROR: Se permitió duplicar full_name',
'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn03_price_not_null()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION

```

```

        CALL p_log_test('RN03', 'RN03: price no puede ser NULL', 'PASS');

CALL p_populate();

INSERT INTO products (description, price, stock) VALUES ('Test', NULL,
10);

CALL p_log_test('RN03', 'ERROR: Se permitió price NULL', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn04_price_nonnegative()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN04', 'RN04: price debe ser >= 0', 'PASS');

    CALL p_populate();

    INSERT INTO products (description, price, stock) VALUES ('Test', -1,
10);

    CALL p_log_test('RN04', 'ERROR: Se permitió price negativo', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn05_stock_nonnegative()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN05', 'RN05: stock debe ser >= 0', 'PASS');

    CALL p_populate();

    INSERT INTO products (description, price, stock) VALUES ('Test', 10,
-5);

    CALL p_log_test('RN05', 'ERROR: Se permitió stock negativo', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn06_amount_not_null()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN06', 'RN06: amount no puede ser NULL', 'PASS');

    CALL p_populate();

    INSERT INTO orders (user_id, product_id, amount, purchase_date)

```

```

VALUES (1, 1, NULL, CURDATE());

CALL p_log_test('RN06', 'ERROR: Se permitió amount NULL', 'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn07_amount_range()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN07', 'RN07: amount debe estar entre 1 y 10',
            'PASS');

    CALL p_populate();

    INSERT INTO orders (user_id, product_id, amount, purchase_date)
        VALUES (1, 1, 20, CURDATE());

    CALL p_log_test('RN07', 'ERROR: Se permitió amount fuera de rango',
        'FAIL');
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE p_test_rn08_august_forbidden()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        CALL p_log_test('RN08', 'RN08: No se permiten compras en agosto',
            'PASS');

    CALL p_populate();

    INSERT INTO orders (user_id, product_id, amount, purchase_date)
        VALUES (1, 1, 2, '2019-08-10');

    CALL p_log_test('RN08', 'ERROR: Se permitió compra en agosto', 'FAIL');
END //
DELIMITER ;

-- =====
-- ORQUESTADOR
-- =====

DELIMITER //
CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
        BEGIN
            CALL p_log_test('POPULATE', 'ERROR: El populate falló. No se
                ejecutaron los tests negativos.', 'ERROR');
            SELECT * FROM test_results ORDER BY execution_time, test_id;

```

```
        SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
    END;

    DELETE FROM test_results;

    CALL p_populate();

    CALL p_test_rn01_user_name_not_null();
    CALL p_test_rn02_user_name_unique();
    CALL p_test_rn03_price_not_null();
    CALL p_test_rn04_price_nonnegative();
    CALL p_test_rn05_stock_nonnegative();
    CALL p_test_rn06_amount_not_null();
    CALL p_test_rn07_amount_range();
    CALL p_test_rn08_august_forbidden();

    SELECT * FROM test_results ORDER BY execution_time, test_id;
    SELECT test_status, COUNT(*) AS total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;
CALL p_run_tests();
```

i Info

Versión PDF disponible